

# Micro-Proyecto No. 1

## Multi-threading and Shared-Memory Ideas

Repositorio: <https://github.com/japeto/multithreaded-enemy-cars>

### Descripción general

A continuación se presentan los lineamientos para el desarrollo de múltiples diseños multihilo (*memoria compartida*) aplicadas al código Enemy Cars (*ver Repositorio*). Se debe implementar un sistema donde los vehículos enemigos se generen y sus posiciones se actualicen de forma asíncrona mediante hilos. El proyecto sigue un diseño cliente servidor, donde el *backend* se encarga de generar los datos de movimiento, mientras que el *frontend* muestra los vehículos enemigos y aplica las posiciones recibidas.

#### Objetivos

Al completar este micro-proyecto, los estudiantes aprenderán a:

- Crear y gestionar la ejecución paralela mediante C++ *std::thread* o *POSIX pthread*.
- Descomponer un problema en tareas independientes (tres variantes).
- Diseñar soluciones basadas en hilos para simulaciones en tiempo real.
- Gestionar datos compartidos entre hilos.

### Diseño 1: Hilos Independientes

En este diseño, cada vehículo enemigo tiene su propio hilo de ejecución. Cada hilo es responsable de actualizar únicamente un vehículo.

None

```
Carro 1 ---> Thread 1
Carro 2 ---> Thread 2
Carro 3 ---> Thread 3
Carro 4 ---> Thread 4
```

### Diseño 2: Hilo único de Actualización

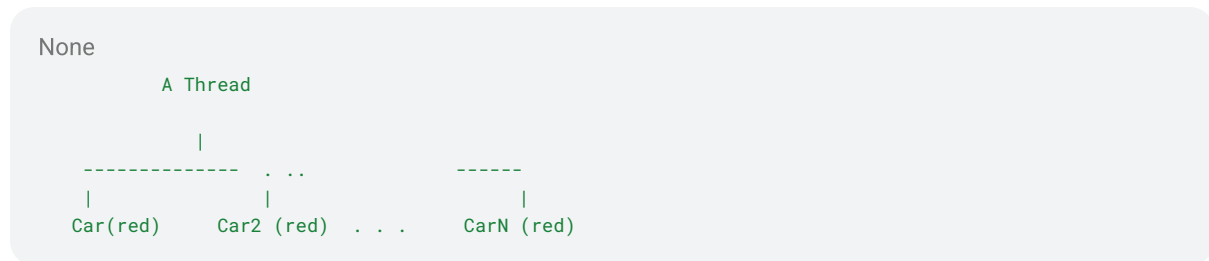
En esta diseño existe un solo hilo encargado de actualizar todos los vehículos.

None

```
      A Thread
      |
-----|-----
      |         |         |
      | Car1    | Car2    | CarN
      |         |         |
```

### Diseño 3: Hilo para cada tipo de vehículo

En esta diseño existe un solo hilo encargado de actualizar todos los vehículos de un tipo, rojos, verdes, negros etc.



### Diseño 4: Hilo asíncrono para vehículos

En este diseño existe un grupo fijo de hilos que ejecutan tareas. Los vehículos no tienen un hilo permanente (evalúe una implementación sin *websockets*). Los hilos toman las solicitudes desde una cola.

### Entrega

Realice el análisis de los diseños, luego responda las siguientes preguntas.

1. ¿Por qué este diseño puede o no aprovechar múltiples núcleos del procesador?
2. ¿Qué tan fácil es agregar nuevos vehículos? ¿Qué nivel de independencia tiene cada vehículo?
3. ¿Qué ocurre si existen miles de vehículos?
4. ¿Incluyó variables compartidas entre hilos para el diseño? ¿Dónde podría ocurrir una condición de carrera?
5. ¿Qué estructuras de datos utilizó para cada diseño?

### Criterios de Evaluación

| Criterio                                                                                                                                         |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Realiza la implementación de cada diseño                                                                                                         | 30 |
| Presenta un análisis de ventajas y desventajas de cada diseño                                                                                    | 20 |
| Responde las preguntas y establece una comparación en términos de rendimiento, complejidad del desarrollo y mantenimiento posterior del sistema. | 20 |
| Justifica las decisiones de implementación de cada diseño, estructura de datos utilizadas, funciones,                                            | 10 |
| Presenta los anteriores criterios en su informe y hace una descripción general de estos en el video                                              | 20 |

### Notas

- Se recomienda el uso de sockets o cualquier otro mecanismo de comunicación entre backend y frontend
- Los vehículos enemigos no deberían compartir posición (noa hay colisión entre enemigos)

## Reflexión

Un sistema concurrente no siempre necesita "*más hilos*" para ser más rápido. El ingeniero de software paralelo debe seleccionar la estrategia que proporcione el mejor equilibrio entre rendimiento, complejidad del desarrollo y mantenimiento posterior del sistema.

### Letra pequeña

- La entrega debe realizarse en las fechas establecidas a través del aula digital.
- No se recibirán trabajos a través de correo electrónico.
- Agregue al informe los enlaces al código, al video o a cualquier otro recurso que haga parte de la entrega.
- La longitud del video debe ser menor o igual a 15 mins, todos los miembros del equipo deben participar.
- Tenga en cuenta los criterios de evaluación descritos en la Rúbrica (Si existen).
- Solo se reciben archivos en formato PDF a través de la plataforma.
- La implementación de cada Diseño debe realizarse en una rama de su repositorio